

Merkle–Damgård Hash Functions

Ahsan Ghafoor

Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
ahgha0019@uit.no

***Index Terms*—Hash functions, Merkle–Damgård, Avalanche effect, collision search, length extension, Merkle tree, SHA-256**

I. ABSTRACT

This report presents the implementation and evaluation of a toy Merkle–Damgård hash function based on the provided assignment template. The work included low-level bit and byte primitives, a compression function, Merkle–Damgård padding, full hashing, a toy length extension attack, and Merkle tree functionality with inclusion proofs. The implementation was tested using the provided self-test and evaluated experimentally. The avalanche experiment was run for 200 trials and compared against SHA-256. Truncated collision searches were performed for 16, 20, and 24-bit outputs, with five independent trials for each size. The toy hash showed an average avalanche effect close to the ideal of half the digest bits changing, and the truncated collision results followed the expected birthday-bound growth trend. A Merkle tree root, proof, and successful verification were also produced. Compared to SHA-256, the toy hash behaved similarly in average avalanche and collision scaling, but SHA-256 was significantly faster and showed more consistent diffusion.

II. INTRODUCTION

A. Problem Definition

This assignment studies how hash functions are built and how their properties can be measured in practice. The task was to complete the provided Python template for a toy Merkle–Damgård hash function and then evaluate its behaviour through experiments. The work also included demonstrating structural weaknesses, such as a toy length extension attack, and using hashing to construct Merkle trees.

The implementation was built directly from the provided precode template by completing the required TODO sections, as required by the assignment.

B. Motivation

Hash functions are central in computer security. They are used for integrity checking, commitments, password storage, digital signatures, and authenticated data structures. Because of this, it is important to understand both how a hash function is built internally and how properties such as diffusion and collision resistance can be evaluated. A toy construction is useful because it makes the internal design visible and easier to test than a production hash such as SHA-256.

C. Objectives

- Implement the required helper functions for bit and byte operations.
- Implement the toy compression function exactly as specified.
- Build the full Merkle–Damgård hash with padding and iteration.
- Measure avalanche behaviour and compare it to SHA-256.
- Search for collisions on truncated digests and compare results to the birthday bound.
- Demonstrate a toy length extension attack.
- Implement Merkle tree root computation, proof generation, and proof verification.
- Compare the toy hash to SHA-256 in terms of avalanche effect, truncated collisions, and performance.

D. Outline

This paper is organized as follows:

- Section III provides the relevant background.
- Section IV describes the implementation and addresses Tasks 1-3.
- Section V explains the evaluation method.
- Section VI presents the results and addresses Tasks 4-8.
- Section VII discusses the findings.
- Section VIII presents possible future work.
- Section IX discloses the use of ChatGPT.
- Section X concludes the report.

III. BACKGROUND

A cryptographic hash function maps an input of arbitrary length to a fixed-size digest. Good hash functions are deterministic, efficient to compute, and are designed so that small input changes produce large output changes. Important security goals include preimage resistance, second-preimage resistance, and collision resistance [1].

The Merkle–Damgård construction builds a hash function from a fixed-input compression function. The message is first padded so that its length becomes a multiple of the block size. The padded message is then processed block by block. Each block updates an internal chaining state, and the final state becomes the digest. This design is simple and influential, but when used carelessly it can also enable length extension attacks [2].

Another important use of hash functions is the Merkle tree. In a Merkle tree, each leaf is hashed, and parent nodes are built by hashing pairs of child hashes. This produces a single root that represents the integrity of the whole dataset. Inclusion proofs allow efficient verification that a given leaf belongs to the tree [3].

IV. SYSTEM DESCRIPTION

A. Idea

The system is a toy 256-bit hash function based on the Merkle-Damgård construction. It processes 64-byte message blocks and maintains an internal state of eight 32-bit words. The main design idea is to combine a compression function with padding and iterative block processing. On top of this hash function, the assignment also builds a toy length extension attack and a Merkle tree implementation.

B. Architecture

The implementation can be divided into five parts. The first is low-level bit and byte primitives. The second is the compression function. Thirdly, Merkle-Damgård padding and full hashing. Fourth, evaluation code for avalanche, collisions, and throughput. Lastly, construction built from hashing, namely length extension and Merkle trees. Helper functions support the compression functions, the compression function supports iterative hashing, and the full hash is then reused by the later experiments and constructions.

C. Design & Implementation

1) *Task 1: Bit and Byte Operations:* The first part of the implementation was a set of helper functions: `u32`, `rotl32`, `rotr32`, `xor_bytes`, `hamming_distance`, and `flip_one_random_bit`. These functions are small, but they are necessary for the correctness of the hash core and the experiments.

What does it mean to mask a value to 32 bits in Python, and why is this necessary? Masking a value to 32 bits means keeping only the lowest 32-bit of the integer, typically using `& 0xFFFFFFFF`. This is necessary because Python integers do not overflow like fixed-width machine integers. In a hash function design based on 32-bit words, arithmetic must behave as if it wraps around at 32-bit. Without masking, the internal state and intermediate values would not match the intended word based behaviour [4].

Explain the difference between bit shifts and bit rotations. Why are rotations common in hash functions? A bit shift moves bits left or right and discards the bits that fall off the edge. A rotation also moves bits, but the bits that fall off one side are wrapped around to the other side. Rotations are common in hash functions because they mix bit positions while preserving all the information in the word. This improves diffusion and avoids the information loss that ordinary shifts create [5].

2) *Task 2: Compression Function:* The compression function updates the internal state using one 64-byte block at a time. It combines message words, round constants, modular addition, XOR, and rotations, and it produces a new 8 word state. Since the assignment required following the template exactly, the implementation matched the given specification without changing the round structure [6].

What role does the compression function play in an iterative hash construction? The compression function is the core transformation in an iterative hash. It takes the current chaining state and one message block as input, mixes them, and produces the next chaining state. By applying this function block by block, the hash can process messages of arbitrary length.

Why do hash designs combine addition, XOR, and rotation instead of using a single operation? Each operation contributes a different kind of mixing. XOR is simple and spreads difference bitwise. Modular addition introduces carries, which create non-linear interactions between bits. Rotation moves bit positions without losing information. When these operations are combined, the result is stronger diffusion and more complex behaviour than if only one operation were used.

3) *Task 3: Build the Hash Function (Merkle-Damgård):* The full toy hash was built by implementing `md_pad`, `toymhash`, and `toymhash_hex`. Padding appends a `0x80` byte, then enough zero bytes to make room for the length field, and finally the original message length encoded as an 8-byte little endian integer. After padding, the message is processed block by block through the compression function, and the final state is returned as a 32-byte digest.

Why is padding required in block-based hash functions? Padding is required because the compression function expects fixed size input blocks. Messages can have arbitrary length, so padding ensures that the message length becomes an exact multiple of the block size. It also makes the end of the message unambiguous.

Why is the message length included in the padding? The message length is included to separate messages that would otherwise produce the same padded structure. It prevents ambiguity between different messages and is part of the standard Merkle-Damgård padding design.

Explain the difference between preimage resistance and collision resistance. Preimage resistance means that, given a digest, it should be hard to find a message that hashes to that digest. Collision resistance means that it should be hard to find two different messages that produce the same digest. The first is about reversing the hash for one target output, while the second is about finding any pair of matching outputs.

V. EVALUATION

The completed implementation was first checked using the provided self-test, which passed successfully. After that, the evaluation focused on four areas: avalanche effect, truncated collision search, throughput, and Merkle tree verification.

For the avalanche experiment, the hash of a random input was compared to the hash of the same input after flipping exactly one bit. The number of changed digest bits was measured using Hamming distance. This experiment was repeated for 200 trials for both the toy hash and SHA-256, and the mean and standard deviation were recorded.

For collision search, the digest was truncated to 16, 20, and 24 bits. For each truncation size, five independent trials were performed. In each trial, random messages were hashed until two different messages produced the same truncated digest. The number of attempts until the first collision was recorded, and the average number of attempts was used for comparison with the expected birthday bound behaviour.

Throughput was measured in hashes per second for both the toy hash and SHA-256 using the provided benchmark command.

For the Merkle tree section, a root was computed from a set of leaves, an inclusion proof was generated for one leaf, and the proof was then verified against the root.

The length extension task was implemented in code as required by the assignment, using the provided stateful interface and extension function. The purpose there was to show how a Merkle-Damgård style hash can be misused when used directly as `hash(secret || message)`.

VI. RESULTS

A. Task 4: Avalanche Effect Experiment

The avalanche experiment was run for 200 trials on both the toy hash and SHA-256. The results are shown in Table I.

TABLE I
AVALANCHE EXPERIMENT RESULTS

Hash	Trials	Mean changed bits	Std. dev.
Toy hash	200	127.71	11.02
SHA-256	200	128.305	8.71

What avalanche behaviour would you expect from a well-designed hash function? For a well-designed 256-bit hash function, changing one input bit should change about half of the output bits on average. Since the digest length is 256 bits, the expected value is around 128 changed bits.

Compare the avalanche results of your toy hash with SHA-256. What do you observe? Both functions produced avalanche behaviour close to the ideal value of 128 changed bits. The toy hash had a mean of 127.71, and SHA-256 had a mean of 128.305. SHA-256 had a lower standard deviation (8.71 vs 11.02), suggesting more uniform diffusion per trial, likely due to its more extensive round structure and message schedule.

B. Task 5: Collision Search on Truncated Digests

The collision search results are summarized in Table II.

Explain the birthday paradox in the context of hash functions. The birthday paradox means that collisions appear sooner than a naive full-space search would suggest. For a

TABLE II
AVERAGE ATTEMPTS UNTIL FIRST TRUNCATED COLLISION

Hash	16 bits	20 bits	24 bits
Toy hash	212.0	589.8	4019.6
SHA-256	327.4	1420.8	4819.0
Expected scale	256	1024	4096

k bit truncated hash, there are 2^k possible outputs, but a collision is expected after about $2^{k/2}$ random inputs. This happens because every new input can match any previously seen output.

Do your experimental results match the expected order of growth? Why or why not? The toy hash tracks the birthday bound very closely (212 -> 589.8 -> 4019.6 follows roughly 4 times growth per 2-bit increase, as expected). SHA-256's results are higher than expected at 20 bits, which we can attribute to randomness with only 5 trials. Another thing worth to note is that both hashes show the same order of growth, confirming the birthday bound holds regardless of the underlying construction.

C. Task 6: Length Extension Attack (Toy Setting)

This assignment also required implementing a toy length extension attack for the insecure usage pattern `toyhash(secret || message)`.

Demonstration. To demonstrate the attack, the toy implementation supports both `toyhash_stateful(message)` and `toyhash_extend(digest, orig_len, extra)`. Given the digest of `secret || message` and a correct guess of the original total length, Eve can compute a valid digest for `secret || message || padding || extra` without knowing the secret. This confirms that the toy Merkle-Damgård construction is vulnerable when misused as `hash(secret || message)`.

Explain what a length extension attack is, in your own words. A length extension attack is an attack against Merkle-Damgård style hashes when they are used directly on `secret || message`. If an attacker knows the digest of the original value and can guess the original total length, the attacker can continue the hashing process from the internal state represented by the digest. This allows the attacker to compute a valid hash for an extended message without knowing the secret itself.

Show how Eve can forge a valid hash for an extended message without knowing the secret. Assume Alice computes `toyhash(secret || message)` and publishes both the message and the digest. Eve does not know `secret`, but Eve can guess the length of `secret || message`. Eve then reconstructs the padding that would have been appended to the original hidden input and uses the published digest as a representation of the internal chaining state. From that state, Eve hashes extra attacker chosen data such as `''&admin=true''`. The result is a valid digest for

`secret || message || padding || extra`

even though Eve never learned the secret. This works because the digest leaks the final chaining state of the original computation.

What is the standard real-world fix for this vulnerability? The standard real-world fix is to use HMAC instead of directly hashing `secret || message`. HMAC is specifically designed to avoid this problem.

D. Task 7: Merkle Tree Hashing

The Merkle tree implementation produced the following root and proof:

- Root: 1e346852a777cb53d41795e08fb8671fb4665ed2a44ef7ff6267a2fb42a759f0
- Proof item 1: 0885ee48448b8322dc759add1386062fa2d7d8c847b2bf8e6a05adc6de9938c0, direction R
- Proof item 2: faf9880408635228c983cecc30089a7050abbe840b5a6e847ff3f9d7fd17a49f, direction L
- Proof item 3: 85f3806d55e2e18b21cb4ebaa593fd442fa1b6ed56dbdd2797bb79a1688cc3ab, direction R
- Verification result: True

Why are Merkle trees useful for protecting the integrity of large datasets? Merkle trees are useful because they allow the integrity of a large dataset to be represented by one root hash. If any leaf changes, the root changes as well. This makes it possible to verify integrity efficiently without reprocessing the entire dataset every time.

What information must an inclusion proof contain, and why? An inclusion proof must contain the sibling hash at each level on the path from the leaf to the root. It must also contain direction information, meaning whether the sibling hash is on the left or the right. This is needed because the verifier must reconstruct the same sequence of concatenations that was used when building the root.

E. Task 8: Comparison with SHA-256

Performance results are shown in Table III.

TABLE III
HASH THROUGHPUT

Hash	Hashes per second
Toy hash	16 766
SHA-256	638 281

SHA-256 was approximately 38.1 times faster than the toy hash in this measurement.

Which metrics show the largest difference between the toy hash and SHA-256? The largest measured difference was performance. The avalanche means were very similar, and the truncated collision results followed the same general birthday bound trend. In contrast, SHA-256 was much faster than the toy hash.

Based on your results, where is your toy hash weakest, and why? The toy hash appears weakest in efficiency and in

overall cryptographic design. Its average avalanche behaviour is good, but its higher standard deviation suggests less uniform diffusion than SHA-256. More importantly, the toy construction is structurally simpler and can be misused in a way that enables length extension. It also performs much more slowly than SHA-256 in this implementation.

VII. DISCUSSION

The results show that a toy Merkle-Damgård construction can already reproduce several important properties of real hash functions. The avalanche experiment was especially interesting because the toy hash reached an average close to the ideal of half the output bits changing. This suggests that even a simplified design can achieve good average diffusion.

At the same time, the comparison with SHA-256 shows why real-world hash design is more than just obtaining a reasonable avalanche score. SHA-256 was more consistent in the avalanche experiment and dramatically faster in the throughput measurement. The truncated collision experiments also remind us that collision behaviour for reduced digest sizes follows the birthday paradox regardless of whether the underlying function is a toy construction or a mature standard. The main difference lies in how much trust can be placed in the full design.

The length extension task highlights an important structural lesson. A Merkle-Damgård hash may be acceptable as a hash function, but it becomes unsafe when directly used as `hash(secret || message)`. This shows that secure use of primitives matters just as much as correct implementation.

VIII. FUTURE WORK

There are several natural extensions to this work. First, the avalanche and collision experiments could be repeated with more trials to reduce variance and give tighter averages. Second, more visual analysis could be added, such as histograms of avalanche distances or plots of collision attempts across trials. Third, the toy hash could be tested against additional misuse cases or modified to explore stronger or weaker diffusion properties. Finally, the Merkle tree implementation could be extended to larger datasets and benchmarked for proof size and verification cost.

IX. AI DECLARATION

AI was used for language clarifications, structural advice, and as a source finder. All technical content, implementation, and final revisions were done by the authors.

X. CONCLUSION

This report presented the implementation and evaluation of a toy Merkle-Damgård hash function. The required helper functions, compression function, padding logic, full hashing, toy length extension functionality, and Merkle tree operations were implemented successfully. The program passed the provided self-test.

The experimental results showed that the toy hash achieved an avalanche effect close to the ideal value of half the digest

bits changing. The truncated collision search followed the expected birthday bound trend, and the Merkle tree proof was verified successfully. When compared with SHA-256, the toy hash showed similar average avalanche behaviour and similar collision-scaling trends, but SHA-256 was more consistent and far faster.

Overall, the assignment gave a practical understanding of how Merkle-Damgård hashing works, how its properties can be measured, and why real-world cryptographic hash functions such as SHA-256 are stronger and more practical than simplified toy constructions.

REFERENCES

- [1] Stack Overflow. *What is a good Hash Function?* 2025. URL: <https://stackoverflow.com/questions/34595/what-is-a-good-hash-function> (visited on 03/20/2026).
- [2] GeeksforGeeks. *Merkle-Damgard Scheme in Cryptography*. 2025. URL: <https://www.geeksforgeeks.org/computer-networks/merkle-damgard-scheme-in-cryptography/> (visited on 03/20/2026).
- [3] GeeksforGeeks. *Introduction to Merkle Tree*. 2024. URL: <https://www.geeksforgeeks.org/dsa/introduction-to-merkle-tree/> (visited on 03/21/2026).
- [4] Stack Overflow. *Why does 32-bit bitmasking work in Python for LeetCode "Single Number II" when Python ints are arbitrary precision?* 2025. URL: <https://stackoverflow.com/questions/79579619/why-does-32-bit-bitmasking-work-in-python-for-leetcode-single-number-ii-when-p> (visited on 03/21/2026).
- [5] X-Engineer. *Bit Shift and Rotation*. URL: <https://x-engineer.org/bit-shift-rotation-scilab/> (visited on 03/22/2026).
- [6] ScienceDirect Topics. *Compression Function – an Overview*. URL: <https://www.sciencedirect.com/topics/computer-science/compression-function> (visited on 03/22/2026).